

1.MySQL中的系统库

1.1.系统库简介

MySQL有几个系统数据库，这几个数据库包含了MySQL服务器运行过程中所需的一些信息以及一些运行状态信息，我们现在稍微了解一下。

```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| lijin     |
| mysql     |
| performance_schema |
| sys       |
+-----+
```

performance_schema

这个数据库里主要保存MySQL服务器运行过程中的一些状态信息，算是对MySQL服务器的一个性能监控。包括统计最近执行了哪些语句，在执行过程的每个阶段都花费了多长时间，内存的使用情况等等信息。

information_schema

这个数据库保存着MySQL服务器维护的所有其他数据库的信息，比如有哪些表、哪些视图、哪些触发器、哪些列、哪些索引。这些是一些描述性信息，称之为元数据。

sys

这个数据库通过视图的形式把information_schema和performance_schema结合起来，让程序员可以更方便的了解MySQL服务器的一些性能信息。

mysql

主要存储了MySQL的用户账户和权限信息，还有一些存储过程、事件的定义信息，一些运行过程中产生的日志信息，一些帮助信息以及时区信息等。

1.2.performance_schema

1.2.1.什么是performance_schema

MySQL的performance_schema 是运行在较低级别的用于监控MySQL Server运行过程中的资源消耗、资源等待等情况的一个功能特性，它具有以下特点。

运行在较低级别：采集的东西相对比较底层，比如磁盘文件、表I/O、表锁等等。

- performance_schema提供了一种在数据库运行时实时检查Server内部执行情况的方法。performance_schema 数据库中的表使用performance_schema存储引擎。该数据库主要关注数据库运行过程中的性能相关数据。
- performance_schema通过监视Server的事件来实现监视其内部执行情况，“事件”就是在Server内部活动中所做的任何事情以及对应的时间消耗，利用这些信息来判断Server中的相关资源被消耗在哪里。一般来说，事件可以是函数调用、操作系统的等待、SQL语句执行的阶段[如SQL语句执行过程中的parsing（解析）或sorting（排序）阶段]或者整个SQL语句的集合。采集事件可以方便地提供Server中的相关存储引擎对磁盘文件、表I/O、表锁等资源的同步调用信息。
- 当前活跃事件、历史事件和事件摘要相关表中记录的信息，能提供某个事件的执行次数、使用时长，进而可用于分析与某个特定线程、特定对象（如mutex或file）相关联的活动。
- performance_schema存储引擎使用Server源代码中的“检测点”来实现事件数据的收集。对于performance_schema实现机制本身的代码没有相关的单独线程来检测，这与其他功能（如复制或事件计划程序）不同。

收集到的事件数据被存储在performance_schema数据库的表中。对于这些表可以使用SELECT语句查询，也可以使用SQL语句更新performance_schema数据库中的表记录（比如动态修改performance_schema的以“setup_”开头的配置表，但要注意，配置表的更改会立即生效，这会影响数据收集）。

- performance_schema的表中数据不会持久化存储在磁盘中，而是保存在内存中，一旦服务器重启，这些数据就会丢失（包括配置表在内的整个performance_schema下的所有数据）。

1.2.2.performance_schema使用

通过上面介绍，相信你对于什么是performance_schema这个问题了解得更清晰了。下面开始介绍performance_schema的使用。

1.2.3.检查当前数据库版本是否支持

performance_schema被视为存储引擎，如果该引擎可用，则应该在

INFORMATION_SCHEMA.ENGINES表或show engines语句的输出中可以看到它的Support字段值为YES，如下所示。

```
select * from INFORMATION_SCHEMA.ENGINES;
show engines;
```

```
mysql> select * from INFORMATION_SCHEMA.ENGINES;
```

ENGINE	SUPPORT	COMMENT	TRANSACTIONS	XA	SAVEPOINTS
InnoDB	DEFAULT	Supports transactions, row-level locking, and foreign keys	YES	YES	YES
MRG_MYISAM	YES	Collection of identical MyISAM tables	NO	NO	NO
MEMORY	YES	Hash based, stored in memory, useful for temporary tables	NO	NO	NO
BLACKHOLE	YES	/dev/null storage engine (anything you write to it disappears)	NO	NO	NO
MyISAM	YES	MyISAM storage engine	NO	NO	NO
CSV	YES	CSV storage engine	NO	NO	NO
ARCHIVE	YES	Archive storage engine	NO	NO	NO
PERFORMANCE_SCHEMA	YES	Performance Schema	NO	NO	NO
FEDERATED	NO	Federated MySQL storage engine	NULL	NULL	NULL

```
mysql> show engines;
```

Engine	Support	Comment	Transactions	XA	Savepoints
InnoDB	DEFAULT	Supports transactions, row-level locking, and foreign keys	YES	YES	YES
MRG_MYISAM	YES	Collection of identical MyISAM tables	NO	NO	NO
MEMORY	YES	Hash based, stored in memory, useful for temporary tables	NO	NO	NO
BLACKHOLE	YES	/dev/null storage engine (anything you write to it disappears)	NO	NO	NO
MyISAM	YES	MyISAM storage engine	NO	NO	NO
CSV	YES	CSV storage engine	NO	NO	NO
ARCHIVE	YES	Archive storage engine	NO	NO	NO
PERFORMANCE_SCHEMA	YES	Performance Schema	NO	NO	NO
FEDERATED	NO	Federated MySQL storage engine	NULL	NULL	NULL

当我们看到performance_schema对应的Support字段值为YES时，就表示当前的数据库版本是支持performance_schema的。但确认了数据库实例支持performance_schema存储引擎就可以使用了吗？NO，很遗憾，performance_schema在MySQL 5.6及之前的版本中默认没有启用，在MySQL 5.7及之后的版本中才修改为默认启用。

mysqld启动之后，通过如下语句查看performance_schema启用是否生效（值为ON表示performance_schema已初始化成功且可以使用了；值为OFF表示在启用performance_schema时发生某些错误，可以查看错误日志进行排查）。

```
show variables like 'performance_schema';
```

```
mysql> show variables like 'performance_schema';
```

Variable_name	Value
performance_schema	ON

（如果要显式启用或关闭performance_schema，则需要使用参数**performance_schema=ON|OFF**来设置，并在my.cnf中进行配置。注意：该参数为只读参数，需要在实例启动之前设置才生效）

现在，可以通过查询INFORMATION_SCHEMA.TABLES表中与performance_schema存储引擎相关的元数据，或者在performance_schema库下使用show tables语句来了解其存在哪些表。

使用show tables语句来查询有哪些performance_schema引擎表。

现在，我们知道了在当前版本中，performance_schema库下一共有87个表，

```
mysql> use performance_schema;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
```

Tables_in_performance_schema
accounts
cond_instances
events_stages_current
events_stages_history
events_stages_history_long
events_stages_summary_by_account_by_event_name
events_stages_summary_by_host_by_event_name

```
| status_by_host  
| status_by_thread  
| status_by_user  
| table_handles  
| table_io_waits_summary_by_index_usage  
| table_io_waits_summary_by_table  
| table_lock_waits_summary_by_table  
| threads  
| user_variables_by_thread  
| users  
| variables_by_thread  
+-----  
87 rows in set (0.00 sec)
```

那么这些表都用于存放什么数据呢？我们如何使用它们来查询数据呢？先来看看这些表是如何分类的。

1.2.4.performance_schema表的分类

performance_schema库下的表可以按照监视的不同维度进行分组，例如：按照不同的数据库对象进行分组、按照不同的事件类型进行分组，或者按照事件类型分组之后，再进一步按照账号、主机、程序、线程、用户等进行细分。

下面介绍按照事件类型分组记录性能事件数据的表。

- 语句事件记录表：记录语句事件信息的表，包括：events_statements_current（当前语句事件表）、events_statements_history（历史语句事件表）、events_statements_history_long（长语句历史事件表）以及一些summary表（聚合后的摘要表）。其中，summary表还可以根据账号（account）、主机（host）、程序（program）、线程（thread）、用户（user）和全局（global）再进行细分。

```
show tables like 'events_statement%';
```

```
mysql> show tables like 'events_statement%';
```

Tables_in_performance_schema (events_statement%)
events_statements_current
events_statements_history
events_statements_history_long
events_statements_summary_by_account_by_event_name
events_statements_summary_by_digest
events_statements_summary_by_host_by_event_name
events_statements_summary_by_program
events_statements_summary_by_thread_by_event_name
events_statements_summary_by_user_by_event_name
events_statements_summary_global_by_event_name

```
10 rows in set (0.00 sec)
```

```
mysql> select * from events_statements_history;
```

THREAD_ID	EVENT_ID	END_EVENT_ID	EVENT_NAME	SOURCE	TIMER_START	TIMER_END	TIMER_WAIT	LOCK_TIME	SQL_TEXT
29	101	101	statement/sql/select		11953188642821000	11953188817727000	174906000	65000000	select * from event
29	102	102	statement/sql/select		11973127948972000	11973128229446000	280474000	64000000	select * from event
29	93	93	statement/sql/show_tables		11777751769145000	11777752031179000	262034000	65000000	show tables
29	94	94	statement/sql/show_tables		11874485015985000	11874485189701000	173716000	56000000	show tables like 'd
29	95	95	statement/sql/show_tables		11874485253312000	11874485346852000	93540000	22000000	show tables like 'd
29	96	96	statement/sql/show_tables		11874485394016000	11874485479588000	85572000	19000000	show tables like 'd
29	97	97	statement/sql/show_tables		11895570224101000	11895570493070000	268969000	62000000	show tables like 'd
29	98	98	statement/sql/select		11923068218536000	11923068414416000	195880000	83000000	select * from event
29	99	99	statement/sql/select		11930713241006000	11930713373677000	132671000	63000000	select * from event
29	100	100	statement/sql/select		11934285112482000	11934285243824000	131342000	64000000	select * from event

- 等待事件记录表：与语句事件记录表类似。

```
show tables like 'events_wait%';
```

```
mysql> show tables like 'events_wait%';
```

Tables_in_performance_schema (events_wait%)
events_waits_current
events_waits_history
events_waits_history_long
events_waits_summary_by_account_by_event_name
events_waits_summary_by_host_by_event_name
events_waits_summary_by_instance
events_waits_summary_by_thread_by_event_name
events_waits_summary_by_user_by_event_name
events_waits_summary_global_by_event_name

```
9 rows in set (0.00 sec)
```

- 阶段事件记录表：记录语句执行阶段事件的表，与语句事件记录表类似。

```
show tables like 'events_stage%';
```

```
mysql> show tables like 'events_stage%';
+-----+
| Tables_in_performance_schema (events_stage%) |
+-----+
| events_stages_current                        |
| events_stages_history                       |
| events_stages_history_long                  |
| events_stages_summary_by_account_by_event_name |
| events_stages_summary_by_host_by_event_name  |
| events_stages_summary_by_thread_by_event_name |
| events_stages_summary_by_user_by_event_name  |
| events_stages_summary_global_by_event_name   |
+-----+
8 rows in set (0.00 sec)
```

- 事务事件记录表：记录与事务相关的事件的表，与语句事件记录表类似。

```
show tables like 'events_transaction%';
```

```
mysql> show tables like 'events_transaction%';
+-----+
| Tables_in_performance_schema (events_transaction%) |
+-----+
| events_transactions_current                    |
| events_transactions_history                   |
| events_transactions_history_long              |
| events_transactions_summary_by_account_by_event_name |
| events_transactions_summary_by_host_by_event_name  |
| events_transactions_summary_by_thread_by_event_name |
| events_transactions_summary_by_user_by_event_name  |
| events_transactions_summary_global_by_event_name   |
+-----+
8 rows in set (0.00 sec)
```

- 监视文件系统层调用的表：

```
show tables like '%file%';
```

```
mysql> show tables like '%file%';
+-----+
| Tables_in_performance_schema (%file%) |
+-----+
| file_instances                          |
| file_summary_by_event_name              |
| file_summary_by_instance                |
+-----+
3 rows in set (0.00 sec)
```

- 监视内存使用的表：

```
show tables like '%memory%';
```

```
mysql> show tables like '%memory%';
+-----+
| Tables_in_performance_schema (%memory%) |
+-----+
| memory_summary_by_account_by_event_name |
| memory_summary_by_host_by_event_name    |
| memory_summary_by_thread_by_event_name   |
| memory_summary_by_user_by_event_name     |
| memory_summary_global_by_event_name      |
+-----+
5 rows in set (0.00 sec)
```

- 动态对performance_schema进行配置的配置表:

```
show tables like '%setup%';
```

```
mysql> show tables like '%setup%';
+-----+
| Tables_in_performance_schema (%setup%) |
+-----+
| setup_actors                           |
| setup_consumers                        |
| setup_instruments                      |
| setup_objects                         |
| setup_timers                          |
+-----+
5 rows in set (0.00 sec)
```

现在，我们已经大概知道了performance_schema中主要表的分类，但如何使用这些表来提供性能事件数据呢？

1.2.5.performance_schema简单配置与使用

当数据库初始化完成并启动时，并非所有的instruments（在采集配置项的配置表中，每一项都有一个开关字段，或为YES，或为NO）和consumers（与采集配置项类似，也有一个对应的事件类型保存表配置项，为YES表示对应的表保存性能数据，为NO表示对应的表不保存性能数据）都启用了，所以默认不会收集所有的事件。

可能你想检测的事件并没有打开，需要进行设置。可以使用如下两条语句打开对应的instruments和consumers，我们以配置监测等待事件数据为例进行说明。

打开等待事件的采集器配置项开关，需要修改setup_instruments 配置表中对应的采集器配置项。

```
update setup_instruments set enabled='yes',timed='yes' where name like 'wait%';
```

```
mysql> update setup_instruments set enabled='yes',timed='yes' where name like 'wait%';
Query OK, 268 rows affected (0.00 sec)
Rows matched: 322 Changed: 268 Warnings: 0
```

打开等待事件的保存表配置项开关，修改setup_consumers 配置表中对应的配置项。

```
update setup_consumers set enabled='yes' where name like 'wait%';
```

```
mysql> update setup_consumers set enabled='yes' where name like 'wait%';
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0 Changed: 0 Warnings: 0
```

配置好之后，我们就可以查看Server当前正在做什么了。可以通过查询events_waits_current表来得知，该表中每个线程只包含一行数据，用于显示每个线程的最新监视事件（正在做的事情）。

_current表中每个线程只保留一条记录，且一旦线程完成工作，该表中就不会再记录该线程的事件信息了。history表中记录每个线程已经执行完成的事件信息，但每个线程的事件信息只记录10条，再多就会被覆盖掉。*history_long表中记录所有线程的事件信息，但总记录数量是10000行，超过会被覆盖掉。

summary表提供所有事件的汇总信息。该组中的表以不同的方式汇总事件数据（如：按用户、按主机、按线程等汇总）。

1.2.6.查看最近执行失败的SQL语句

使用代码对数据库的某些操作（比如：使用Java的ORM框架操作数据库）报出语法错误，但是代码并没有记录SQL语句文本的功能，在MySQL数据库层能否查看到具体的SQL语句文本，看看是否哪里写错了？这个时候，大多数人首先想到的就是去查看错误日志。很遗憾，对于SQL语句的语法错误，错误日志并不会记录。

实际上，在performance_schema的语句事件记录表中针对每一条语句的执行状态都记录了较为详细的信息，例如：events_statements表和events_statements_summary_by_digest表（events_statements表记录了语句所有的执行错误信息，而events_statements_summary_by_digest表只记录了语句在执行过程中发生错误的语句记录统计信息，不记录具体的错误类型，例如：不记录语法错误类的信息）。下面看看如何使用这两个表查询语句发生错误的语句信息。

首先，我们模拟一条语法错误的SQL语句，使用events_statements_history_long表或events_statements_history表查询发生语法错误的SQL语句：

```
Database changed
mysql> select * from;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version 1
```

然后，查询events_statements_history表中错误号为1064的记录

```
select * from events_statements_history where mysql_errno=1064\G
```

```
mysql> select * from events_statements_history where mysql_errno=1064\G
***** 1. row *****
  THREAD_ID: 30
  EVENT_ID: 8
  END_EVENT_ID: 8
  EVENT_NAME: statement/sql/error
  SOURCE:
  TIMER_START: 12740369772712000
  TIMER_END: 12740369820500000
  TIMER_WAIT: 47788000
  LOCK_TIME: 0
  SQL_TEXT: select * from
  DIGEST: NULL
  DIGEST_TEXT: NULL
  CURRENT_SCHEMA: lijin
  OBJECT_TYPE: NULL
  OBJECT_SCHEMA: NULL
  OBJECT_NAME: NULL
  OBJECT_INSTANCE_BEGIN: NULL
  MYSQL_ERRNO: 1064
  RETURNED_SQLSTATE: 42000
  MESSAGE_TEXT: You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use
  ERRORS: 1
```

如果不知道错误号是多少，可以查询发生错误次数不为0的语句记录，在里边找到SQL_TEXT和MESSAGE_TEXT字段（提示信息为语法错误的就是它）。

1.2.7.查看最近的事务执行信息

我们可以通过慢查询日志查询到一条语句的执行总时长，但是如果数据库中存在着一些大事务在执行过程中回滚了，或者在执行过程中异常中止，这个时候慢查询日志就爱莫能助了，这时我们可以借助performance_schema的events_transactions_*表来查看与事务相关的记录，在这些表中详细记录了是否有事务被回滚、活跃（长时间未提交的事务也属于活跃事务）或已提交等信息。

首先需要进行配置启用，事务事件默认并未启用

```
update setup_instruments set enabled='yes',timed='yes' where name like 'transaction%';
```

```
update setup_consumers set enabled='yes' where name like '%transaction%';
```

```
mysql> update setup_instruments set enabled='yes',timed='yes' where name like 'transaction%';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> update setup_consumers set enabled='yes' where name like '%transaction%';
Query OK, 3 rows affected (0.00 sec)
Rows matched: 3 Changed: 3 Warnings: 0
```

现在我们开启一个新会话（会话2）用于执行事务，并模拟事务回滚。

```
mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> insert test values(100);
Query OK, 1 row affected (0.00 sec)
```

查询活跃事务，活跃事务表示当前正在执行的事务事件，需要从events_transactions_current表中查询。

下图中可以看到有一条记录，代表当前活跃的事务事件。

```
mysql> select * from events_transactions_current\G
***** 1. row *****
      THREAD_ID: 30
      EVENT_ID: 16
      END_EVENT_ID: NULL
      EVENT_NAME: transaction
      STATE: ACTIVE
      TRX_ID: NULL
      GTID: AUTOMATIC
      XID_FORMAT_ID: NULL
      XID_GTRID: NULL
      XID_BQUAL: NULL
      XA_STATE: NULL
      SOURCE:
      TIMER_START: 13732439534275000
      TIMER_END: 14082463277328000
      TIMER_WAIT: 350023743053000
      ACCESS_MODE: READ WRITE
      ISOLATION_LEVEL: REPEATABLE READ
      AUTOCOMMIT: NO
      NUMBER_OF_SAVEPOINTS: 0
      NUMBER_OF_ROLLBACK_TO_SAVEPOINT: 0
      NUMBER_OF_RELEASE_SAVEPOINT: 0
      OBJECT_INSTANCE_BEGIN: NULL
      NESTING_EVENT_ID: 15
      NESTING_EVENT_TYPE: STATEMENT
1 row in set (0.00 sec)
```

会话2中回滚事务：

```
mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> insert test values(100);
Query OK, 1 row affected (0.00 sec)

mysql> rollback;
Query OK, 0 rows affected (0.00 sec)
```

查询事务事件当前表 (events_transactions_current) 和事务事件历史记录表 (events_transactions_history)

可以看到在两表中都记录了一行事务事件信息，线程ID为30的线程执行了一个事务，事务状态为ROLLED BACK。

```
mysql> select * from events_transactions_current\G
***** 1. row *****
      THREAD_ID: 30
      EVENT_ID: 16
      END_EVENT_ID: 18
      EVENT_NAME: transaction
      STATE: ROLLED BACK
      TRX_ID: NULL
      GTID: AUTOMATIC
      XID_FORMAT_ID: NULL
      XID_GTRID: NULL
      XID_BQUAL: NULL
      XA_STATE: NULL
      SOURCE:
      TIMER_START: 13732439534275000
      TIMER_END: 14246952799585000
      TIMER_WAIT: 514513265310000
      ACCESS_MODE: READ WRITE
      ISOLATION_LEVEL: REPEATABLE READ
      AUTOCOMMIT: NO
      NUMBER_OF_SAVEPOINTS: 0
      NUMBER_OF_ROLLBACK_TO_SAVEPOINT: 0
      NUMBER_OF_RELEASE_SAVEPOINT: 0
      OBJECT_INSTANCE_BEGIN: NULL
      NESTING_EVENT_ID: 15
      NESTING_EVENT_TYPE: STATEMENT
1 row in set (0.00 sec)
```

```
mysql> select * from events_transactions_history\G
```

```
***** 2. row *****
      THREAD_ID: 30
      EVENT_ID: 16
      END_EVENT_ID: 18
      EVENT_NAME: transaction
      STATE: ROLLED BACK
      TRX_ID: NULL
      GTID: AUTOMATIC
      XID_FORMAT_ID: NULL
      XID_GTRID: NULL
      XID_BQUAL: NULL
      XA_STATE: NULL
      SOURCE:
      TIMER_START: 13732439534275000
      TIMER_END: 14246952799585000
      TIMER_WAIT: 514513265310000
      ACCESS_MODE: READ WRITE
      ISOLATION_LEVEL: REPEATABLE READ
      AUTOCOMMIT: NO
      NUMBER_OF_SAVEPOINTS: 0
      NUMBER_OF_ROLLBACK_TO_SAVEPOINT: 0
      NUMBER_OF_RELEASE_SAVEPOINT: 0
      OBJECT_INSTANCE_BEGIN: NULL
      NESTING_EVENT_ID: 15
      NESTING_EVENT_TYPE: STATEMENT
2 rows in set (0.00 sec)
```

但是当我们关闭会话2以后，事务事件当前表中（events_transactions_current）的记录就消失了。

```
mysql> select * from events_transactions_current\G
Empty set (0.00 sec)
```

要查询的话需要去 (events_transactions_history_long) 表中查

```
mysql> select * from events_transactions_history\G
Empty set (0.00 sec)
```

```
mysql> select * from events_transactions_history_long\G
***** 1 row *****
```

```
***** 3 row *****
      THREAD_ID: 30
      EVENT_ID: 16
      END_EVENT_ID: 18
      EVENT_NAME: transaction
      STATE: ROLLED BACK
      TRX_ID: NULL
      GTID: AUTOMATIC
      XID_FORMAT_ID: NULL
      XID_GTRID: NULL
      XID_BQUAL: NULL
      XA_STATE: NULL
      SOURCE:
      TIMER_START: 13732439534275000
      TIMER_END: 14246952799585000
      TIMER_WAIT: 514513265310000
      ACCESS_MODE: READ WRITE
      ISOLATION_LEVEL: REPEATABLE READ
      AUTOCOMMIT: NO
      NUMBER_OF_SAVEPOINTS: 0
      NUMBER_OF_ROLLBACK_TO_SAVEPOINT: 0
      NUMBER_OF_RELEASE_SAVEPOINT: 0
      OBJECT_INSTANCE_BEGIN: NULL
      NESTING_EVENT_ID: 15
      NESTING_EVENT_TYPE: STATEMENT
3 rows in set (0.00 sec)
```

1.2.8.小结

当然performance_schema的用途不止我们上面说到过的这些，它还能提供比如查看SQL语句执行阶段和进度信息、MySQL集群下复制功能查看复制报错详情等等。

具体可以参考官网：[MySQL :: MySQL 5.7 Reference Manual :: 25 MySQL Performance Schema](#)

1.3.sys系统库

1.3.1.sys使用须知

sys系统库支持MySQL 5.6或更高版本，不支持MySQL 5.5.x及以下版本。

sys系统库通常都是提供给专业的DBA人员排查一些特定问题使用的，其下所涉及的各项查询或多或少都会对性能有一定的影响。

因为sys系统库提供了一些代替直接访问performance_schema的视图，所以必须启用performance_schema（将performance_schema系统参数设置为ON），sys系统库的大部分功能才能正常使用。

同时要完全访问sys系统库，用户必须具有以下数据库的管理员权限。

如果要充分使用sys系统库的功能，则必须启用某些performance_schema的功能。比如：

启用所有的wait instruments：

```
CALL sys.ps_setup_enable_instrument('wait');
```

```
mysql> CALL sys.ps_setup_enable_instrument('wait');
+-----+
| summary |
+-----+
| Enabled 49 instruments |
+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

启用所有事件类型的current表：

```
CALL sys.ps_setup_enable_consumer('current');
```

```
mysql> CALL sys.ps_setup_enable_consumer('current');
+-----+
| summary |
+-----+
| Enabled 2 consumers |
+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

注意：performance_schema的默认配置就可以满足sys系统库的大部分数据收集功能。启用所有需要功能会对性能产生一定的影响，因此最好仅启用所需的配置。

1.3.2.sys系统库使用

如果使用了USE语句切换默认数据库，那么就可以直接使用sys系统库下的视图进行查询，就像查询某个库下的表一样操作。也可以使用db_name.view_name、db_name.procedure_name、db_name.func_name等方式，在不指定默认数据库的情况下访问sys系统库中的对象（这叫作名称限定对象引用）。

```
mysql> use sys;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> select * from version;
+-----+-----+
| sys_version | mysql_version |
+-----+-----+
| 1.5.2       | 5.7.38-log    |
+-----+-----+
1 row in set (0.00 sec)

mysql> select * from lijin.test
-> ;
Empty set (0.00 sec)
```

在sys系统库下包含很多视图，它们以各种方式对performance_schema表进行聚合计算展示。这些视图大部分是成对出现的，两个视图名称相同，但有一个视图是带 x \$前缀的.\$

host_summary_by_file_io和 x\$host_summary_by_file_io

代表按照主机进行汇总统计的文件I/O性能数据，两个视图访问的数据源是相同的，但是在创建视图的语句中，不带x\$前缀的视图显示的是相关数值经过单位换算后的数据（单位是毫秒、秒、分钟、小时、天等），带 x\$ 前缀的视图显示的是原始的数据（单位是皮秒）。

```
mysql> show tables;
+-----+
| Tables_in_sys |
+-----+
| host_summary |
| host_summary_by_file_io |
| host_summary_by_file_io_type |
| host_summary_by_stages |
| host_summary_by_statement_latency |
| host_summary_by_statement_type |
| innodb_buffer_stats_by_schema |
| innodb_buffer_stats_by_table |
| innodb_lock_waits |
| io_by_thread_by_latency |
| io_global_by_file_by_bytes |
| io_global_by_file_by_latency |
| io_global_by_wait_by_bytes |
| io_global_by_wait_by_latency |
| latest_file_io |
+-----+
```

```

| waits_by_user_by_latency
| waits_global_by_latency
| x$host_summary
| x$host_summary_by_file_io
| x$host_summary_by_file_io_type
| x$host_summary_by_stages
| x$host_summary_by_statement_latency
| x$host_summary_by_statement_type
| x$innodb_buffer_stats_by_schema
| x$innodb_buffer_stats_by_table
| x$innodb_lock_waits
| x$io_by_thread_by_latency
| x$io_global_by_file_by_bytes
| x$io_global_by_file_by_latency
| x$io_global_by_wait_by_bytes
| x$io_global_by_wait_by_latency

```

```

mysql> select * from host_summary\G
***** 1. row *****
      host: localhost
    statements: 413
  statement_latency: 145.80 ms
statement_avg_latency: 353.02 us
      table_scans: 59
        file_ios: 828
   file_io_latency: 74.61 ms
current_connections: 1
  total_connections: 4
      unique_users: 1
    current_memory: 0 bytes
total_memory_allocated: 0 bytes
1 row in set (0.01 sec)

mysql> select * from x$host_summary\G
***** 1. row *****
      host: localhost
    statements: 453
  statement_latency: 154133649000
statement_avg_latency: 340250880.7947
      table_scans: 60
        file_ios: 828
   file_io_latency: 74606363908
current_connections: 1
  total_connections: 4
      unique_users: 1
    current_memory: 0
total_memory_allocated: 0
1 row in set (0.01 sec)

```

1.3.3.查看慢SQL语句慢在哪里

如果我们频繁地在慢查询日志中发现某个语句执行缓慢，且在表结构、索引结构、统计信息中都无法找出原因时，则可以利用sys系统库中的杀手锏：sys.session视图结合performance_schema的等待事件来找出症结所在。那么session视图有什么用呢？使用它可以查看当前用户会话的进程列表信息，看看当前进程到底再干什么，注意，这个视图在MySQL 5.7.9中才出现。

```
mysql> select version();
+-----+
| version() |
+-----+
| 5.7.38-log |
+-----+
1 row in set (0.00 sec)
```

首先需要启用与等待事件相关功能:

```
call sys.ps_setup_enable_instrument('wait');
call sys.ps_setup_enable_consumer('wait');
```

```
mysql> call sys.ps_setup_enable_instrument('wait');
+-----+
| summary |
+-----+
| Enabled 8 instruments |
+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

mysql> call sys.ps_setup_enable_consumer('wait');
+-----+
| summary |
+-----+
| Enabled 2 consumers |
+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

然后模拟一下:

一个session中执行

```
select sleep(30);
```

另外一个session中在sys库中查询:

```
select * from session where command='query' and conn_id !=connection_id()\G
```

```
mysql> select sleep(30);
```

```
mysql> select * from session where command='query' and conn_id !=connection_id()\G
***** 1. row *****
      thd_id: 36
      conn_id: 10
      user: root@localhost
      db: NULL
      command: Query
      state: User sleep
      time: 3
      current_statement: select sleep(30)
      statement_latency: 3.35 s
      progress: NULL
      lock_latency: 0 ps
      rows_examined: 0
      rows_sent: 0
      rows_affected: 0
      tmp_tables: 0
      tmp_disk_tables: 0
      full_scan: NO
      last_statement: NULL
last_statement_latency: NULL
      current_memory: 0 bytes
      last_wait: wait/synch/cond/sql/Item_func_sleep::cond
      last_wait_latency: Still Waiting
      source:
      trx_latency: NULL
```

查询表的增、删、改、查数据量和I/O耗时统计

```
select * from schema_table_statistics_with_buffer\G
```

```
mysql> select * from schema_table_statistics_with_buffer\G
***** 1. row *****
      table_schema: sys
      table_name: sys_config
      rows_fetched: 1
      fetch_latency: 546.99 us
      rows_inserted: 0
      insert_latency: 0 ps
      rows_updated: 0
      update_latency: 0 ps
      rows_deleted: 0
      delete_latency: 0 ps
      io_read_requests: 12
      io_read: 114.44 KiB
      io_read_latency: 1.96 ms
      io_write_requests: 0
      io_write: 0 bytes
      io_write_latency: 0 ps
      io_misc_requests: 13
      io_misc_latency: 972.44 us
      innodb_buffer_allocated: 16.00 KiB
      innodb_buffer_data: 338 bytes
      innodb_buffer_free: 15.67 KiB
      innodb_buffer_pages: 1
      innodb_buffer_pages_hashed: 0
      innodb_buffer_pages_old: 0
      innodb_buffer_rows_cached: 6
***** 2. row *****
      table_schema: information
```

1.3.4.小结

除此之外，通过sys还可以查询查看InnoDB缓冲池中的热点数据、查看是否有事务锁等待、查看未使用的，冗余索引、查看哪些语句使用了全表扫描等等。

具体可以参考官网：[MySQL :: MySQL 5.7 Reference Manual :: 26 MySQL sys Schema](#)

1.4.information_schema

1.4.1.什么是information_schema

information_schema提供了对数据库元数据、统计信息以及有关MySQL Server信息的访问（例如：数据库名或表名、字段的数据类型和访问权限等）。该库中保存的信息也可以称为MySQL的数据字典或系统目录。

在每个MySQL实例中都有一个独立的information_schema，用来存储MySQL实例中所有其他数据库的基本信息。information_schema库下包含多个只读表（非持久表），所以在磁盘中的数据目录下没有对应的关联文件，且不能对这些表设置触发器。虽然在查询时可以使用USE语句将默认数据库设置为information_schema，但该库下的所有表是只读的，不能执行INSERT、UPDATE、DELETE等数据变更操作。

针对information_schema下的表的查询操作可以替代一些SHOW查询语句（例如：SHOW DATABASES、SHOW TABLES等）。

注意：根据MySQL版本的不同，表的个数和存放是有所不同的。在MySQL 5.6版本中总共有59个表，在MySQL 5.7版本中，该schema下总共有61个表，

```
mysql> show tables;
```

+-----+-----+	
Tables_in_information_schema	
+-----+-----+	
CHARACTER_SETS	
COLLATIONS	
COLLATION_CHARACTER_SET_APPLICABILITY	
COLUMNS	
COLUMN_PRIVILEGES	
ENGINES	
EVENTS	
FILES	
GLOBAL_STATUS	
GLOBAL_VARIABLES	
KEY_COLUMN_USAGE	
OPTIMIZER_TRACE	
PARAMETERS	
PARTITIONS	
PLUGINS	
PROCESSLIST	
PROFILING	
REFERENTIAL_CONSTRAINTS	
ROUTINES	
SCHEMATA	
SCHEMA_PRIVILEGES	
SESSION_STATUS	
SESSION_VARIABLES	
SESSION_VARIABLES	
STATISTICS	
TABLES	
TABLESPACES	
TABLE_CONSTRAINTS	
TABLE_PRIVILEGES	
TRIGGERS	
USER_PRIVILEGES	
VIEWS	
INNODB_LOCKS	
INNODB_TRX	
INNODB_SYS_DATAFILES	
INNODB_FT_CONFIG	
INNODB_SYS_VIRTUAL	
INNODB_CMP	
INNODB_FT_BEING_DELETED	
INNODB_CMP_RESET	
INNODB_CMP_PER_INDEX	
INNODB_CMPMEM_RESET	

```

| INNODB_CMPMEM_RESET
| INNODB_FT_DELETED
| INNODB_BUFFER_PAGE_LRU
| INNODB_LOCK_WAITS
| INNODB_TEMP_TABLE_INFO
| INNODB_SYS_INDEXES
| INNODB_SYS_TABLES
| INNODB_SYS_FIELDS
| INNODB_CMP_PER_INDEX_RESET
| INNODB_BUFFER_PAGE
| INNODB_FT_DEFAULT_STOPWORD
| INNODB_FT_INDEX_TABLE
| INNODB_FT_INDEX_CACHE
| INNODB_SYS_TABLESPACES
| INNODB_METRICS
| INNODB_SYS_FOREIGN_COLS
| INNODB_CMPMEM
| INNODB_BUFFER_POOL_STATS
| INNODB_SYS_COLUMNS
| INNODB_SYS_FOREIGN
| INNODB_SYS_TABLESTATS
+-----+
61 rows in set (0.00 sec)

```

在MySQL 8.0版本中，该schema下的数据字典表（包含部分原Memory引擎临时表）都迁移到了mysql schema下，且在mysql schema下这些数据字典表被隐藏，无法直接访问，需要通过information_schema下的同名表进行访问。

information_schema下的所有表使用的都是Memory和InnoDB存储引擎，且都是临时表，不是持久表，在数据库重启之后这些数据会丢失。在MySQL的4个系统库中，information_schema也是唯一一个在文件系统上没有对应库表的目录和文件的系统库。

1.4.2.information_schema表分类

Server层的统计信息字典表

(1) COLUMNS

- 提供查询表中的列（字段）信息。

(2) KEY_COLUMN_USAGE

- 提供查询哪些索引列存在约束条件。
- 该表中的信息包含主键、唯一索引、外键等约束信息，例如：所在的库表列名、引用的库表列名等。该表中的信息与TABLE_CONSTRAINTS表中记录的信息有些类似，但TABLE_CONSTRAINTS表中没有记录约束引用的库表列信息，而KEY_COLUMN_USAGE表中却记录了TABLE_CONSTRAINTS表中所没有的约束类型。

(3) REFERENTIAL_CONSTRAINTS

- 提供查询关于外键约束的一些信息。

(4) STATISTICS

- 提供查询关于索引的一些统计信息，一个索引对应一行记录。

(5) TABLE_CONSTRAINTS

- 提供查询与表相关的约束信息。

(6) FILES

- 提供查询与MySQL的数据表空间文件相关的信息。

(7) ENGINES

- 提供查询MySQL Server支持的引擎相关信息。

(8) TABLESPACES

- 提供查询关于活跃表空间的相关信息（主要记录的是NDB存储引擎的表空间信息）。
- 注意：该表不提供有关InnoDB存储引擎的表空间信息。对于InnoDB表空间的元数据信息，请查询INNODB_SYS_TABLESPACES表和INNODB_SYS_DATAFILES表。另外，从MySQL 5.7.8开始，INFORMATION_SCHEMA.FILES表也提供查询InnoDB表空间的元数据信息。

(9) SCHEMATA

- 提供查询MySQL Server中的数据库列表信息，一个schema就代表一个数据库。

Server层的表级别对象字典表

(1) VIEWS

- 提供查询数据库中的视图相关信息。查询该表的账户需要拥有show view权限。

(2) TRIGGERS

- 提供查询关于某个数据库下的触发器相关信息。

(3) TABLES

- 提供查询与数据库内的表相关的基本信息。

(4) ROUTINES

- 提供查询关于存储过程和存储函数的信息（不包括用户自定义函数）。该表中的信息与mysql.proc中记录的信息相对应（如果该表中有值的话）。

(5) PARTITIONS

- 提供查询关于分区表的信息。

(6) EVENTS

- 提供查询与计划任务事件相关的信息。

(7) PARAMETERS

- 提供有关存储过程和函数的参数信息，以及有关存储函数的返回值信息。这些参数信息与mysql.proc表中的param_list列记录的内容类似。

Server层的混杂信息字典表

(1) GLOBAL_STATUS、GLOBAL_VARIABLES、SESSION_STATUS、SESSION_VARIABLES

- 提供查询全局、会话级别的状态变量与系统变量信息。

(2) OPTIMIZER_TRACE

- 提供优化程序跟踪功能产生的信息。
- 跟踪功能默认是关闭的，使用optimizer_trace系统变量启用跟踪功能。如果开启该功能，则每个会话只能跟踪它自己执行的语句，不能看到其他会话执行的语句，且每个会话只能记录最后一条跟踪的SQL语句。

(3) PLUGINS

- 提供查询关于MySQL Server支持哪些插件的信息。

(4) PROCESSLIST

- 提供查询一些关于线程运行过程中的状态信息。

(5) PROFILING

- 提供查询关于语句性能分析的信息。其记录内容对应于SHOW PROFILES和SHOW PROFILE语句产生的信息。该表只有在会话变量 profiling=1时才会记录语句性能分析信息，否则该表不记录。
- 注意：从MySQL 5.7.2开始，此表不再推荐使用，在未来的MySQL版本中删除，改用Performance Schema代替。

(6) CHARACTER_SETS

- 提供查询MySQL Server支持的可用字符集。

(7) COLLATIONS

- 提供查询MySQL Server支持的可用校对规则。

(8) COLLATION_CHARACTER_SET_APPLICABILITY

- 提供查询MySQL Server中哪种字符集适用于什么校对规则。查询结果集相当于从SHOW COLLATION获得的结果集的前两个字段值。目前并没有发现该表有太大的作用。

(9) COLUMN_PRIVILEGES

- 提供查询关于列（字段）的权限信息，表中的内容来自mysql.column_priv列权限表（需要针对一个表的列单独授权之后才会有内容）。

(10) SCHEMA_PRIVILEGES

- 提供查询关于库级别的权限信息，每种类型的库级别权限记录一行信息，该表中的信息来自mysql.db表。

(11) TABLE_PRIVILEGES

- 提供查询关于表级别的权限信息，该表中的内容来自mysql.tables_priv表。

(12) USER_PRIVILEGES

- 提供查询全局权限的信息，该表中的信息来自mysql.user表。

10.2.4 InnoDB层的系统字典表

(1) INNODB_SYS_DATAFILES

- 提供查询InnoDB所有表空间类型文件的元数据（内部使用的表空间ID和表空间文件的路径信息），包括独立表空间、常规表空间、系统表空间、临时表空间和undo空间（如果开启了独立undo空间的话）。

- 该表中的信息等同于InnoDB数据字典内部SYS_DATAFILES表的信息。

(2) INNODB_SYS_VIRTUAL

- 提供查询有关InnoDB虚拟生成列和与之关联的列的元数据信息，等同于InnoDB数据字典内部SYS_VIRTUAL表的信息。该表中展示的行信息是与虚拟生成列相关联列的每个列的信息。

(3) INNODB_SYS_INDEXES

- 提供查询有关InnoDB索引的元数据信息，等同于InnoDB数据字典内部SYS_INDEXES表中的信息。

(4) INNODB_SYS_TABLES

- 提供查询有关InnoDB表的元数据信息，等同于InnoDB数据字典内部SYS_TABLES表的信息。

(5) INNODB_SYS_FIELDS

- 提供查询有关InnoDB索引键列（字段）的元数据信息，等同于InnoDB数据字典内部SYS_FIELDS表的信息。

(6) INNODB_SYS_TABLESPACES

- 提供查询有关InnoDB独立表空间和普通表空间的元数据信息（也包含了全文索引表空间），等同于InnoDB数据字典内部SYS_TABLESPACES表的信息。

(7) INNODB_SYS_FOREIGN_COLS

- 提供查询有关InnoDB外键列的状态信息，等同于InnoDB数据字典内部SYS_FOREIGN_COLS表的信息。

(8) INNODB_SYS_COLUMNS

- 提供查询有关InnoDB表列的元数据信息，等同于InnoDB数据字典内部SYS_COLUMNS表的信息。

(9) INNODB_SYS_FOREIGN

- 提供查询有关InnoDB外键的元数据信息，等同于InnoDB数据字典内部SYS_FOREIGN表的信息。

(10) INNODB_SYS_TABLESTATS

- 提供查询有关InnoDB表的较低级别的状态信息视图。MySQL优化器会使用这些统计信息数据来计算并确定在查询InnoDB表时要使用哪个索引。这些信息保存在内存中的数据结构中，与存储在磁盘上的数据无对应关系。在InnoDB内部也无对应的系统表。

InnoDB层的锁、事务、统计信息字典表

(1) INNODB_LOCKS

- 提供查询InnoDB引擎中事务正在请求的且同时被其他事务阻塞的锁信息（即没有发生不同事务之间锁等待的锁信息，在这里是查看不到的。例如，当只有一个事务时，无法查看到该事务所加的锁信息）。该表中的内容可用于诊断高并发下的锁争用信息。

(2) INNODB_TRX

- 提供查询当前在InnoDB引擎中执行的每个事务（不包括只读事务）的信息，包括事务是否正在等待锁、事务什么时间点开始，以及事务正在执行的SQL语句文本信息等（如果有SQL语句的话）。

(3) INNODB_BUFFER_PAGE_LRU

- 提供查询缓冲池中的页面信息。与INNODB_BUFFER_PAGE表不同，INNODB_BUFFER_PAGE_LRU表保存有关InnoDB缓冲池中的页如何进入LRU链表，以及在缓冲池不够用时确定需要从中逐出哪些页的信息。

(4) INNODB_LOCK_WAITS

- 提供查询InnoDB事务的锁等待信息。如果查询该表为空，则表示无锁等待信息；如果查询该表中有记录，则说明存在锁等待，表中的每一行记录表示一个锁等待关系。在一个锁等待关系中包含：一个等待锁（即，正在请求获得锁）的事务及其正在等待的锁等信息、一个持有锁（这里指的是发生锁等待事务正在请求的锁）的事务及其所持有的锁等信息。

(5) INNODB_TEMP_TABLE_INFO

- 提供查询有关在InnoDB实例中当前处于活动状态的用户（只对已建立连接的用户有效，断开的用户连接对应的临时表会被自动删除）创建的InnoDB临时表的信息。它不提供查询优化器使用的内部InnoDB临时表的信息。该表在首次查询时创建。

(6) INNODB_BUFFER_PAGE

- 提供查询关于缓冲池中的页相关信息。

(7) INNODB_METRICS

- 提供查询InnoDB更为详细的性能信息，是对InnoDB的performance_schema的补充。通过对该表的查询，可用于检查InnoDB的整体健康状况，也可用于诊断性能瓶颈、资源短缺和应用程序的问题等。

(8) INNODB_BUFFER_POOL_STATS

- 提供查询一些InnoDB缓冲池中的状态信息，该表中记录的信息与SHOW ENGINEINNODB STATUS语句输出的缓冲池统计部分信息类似。另外，InnoDB缓冲池的一些状态变量也提供了部分相同的值。

InnoDB层的全文索引字典表

(1) INNODB_FT_CONFIG

(2) INNODB_FT_BEING_DELETED

(3) INNODB_FT_DELETED

(4) INNODB_FT_DEFAULT_STOPWORD

(5) INNODB_FT_INDEX_TABLE

InnoDB层的压缩相关字典表

(1) INNODB_CMP和INNODB_CMP_RESET

- 这两个表中的数据包含了与压缩的InnoDB表页有关的操作状态信息。表中记录的数据为测量数据库中的InnoDB表压缩的有效性提供参考。

(2) INNODB_CMP_PER_INDEX和INNODB_CMP_PER_INDEX_RESET

- 这两个表中记录了与InnoDB压缩表数据和索引相关的操作状态信息，对数据库、表、索引的每个组合使用不同的统计信息，以便为评估特定表的压缩性能和实用性提供参考数据。

(3) INNODB_CMPMEM和INNODB_CMPMEM_RESET

- 这两个表中记录了InnoDB缓冲池中压缩页的状态信息，为测量数据库中InnoDB表压缩的有效性提供参考。

1.4.3.information_schema应用

查看索引列的信息

INNODB_SYS_FIELDS表提供查询有关InnoDB索引列（字段）的元数据信息，等同于InnoDB数据字典中SYS_FIELDS表的信息。

INNODB_SYS_INDEXES表提供查询有关InnoDB索引的元数据信息，等同于InnoDB数据字典内部SYS_INDEXES表中的信息。

INNODB_SYS_TABLES表提供查询有关InnoDB表的元数据信息，等同于InnoDB数据字典中SYS_TABLES表的信息。

假设需要查询lijin库下的InnoDB表order_exp的索引列名称、组成和索引列顺序等相关信息，

lijin

表

视图

函数

事件

查询

报表

备份

mail

mysql

performance_sch

rocket_shop

sakila

sys

表

sys_config

视图

order_exp @lijin (MySQL5.7) - 表

文件 编辑 窗口 帮助

新建 保存 另存为 添加栏位 插入栏位 删除栏位 主键 上移 下移

栏位 索引 外键 触发器 选项 注释 SQL 预览

名	类型	长度	小数点	不是 null	
id	bigint	22	0	☒	1
order_no	varchar	50	0	☒	
order_note	varchar	100	0	☒	
insert_time	datetime	0	0	☒	
expire_duration	bigint	22	0	☒	
expire_time	datetime	0	0	☒	
order_status	smallint	6	0	☒	

order_exp @lijin (MySQL5.7) - 表

文件 编辑 窗口 帮助

新建 保存 另存为 添加索引 删除索引

栏位 索引 外键 触发器 选项 注释 SQL 预览

名	栏位	索引类型	索引方法
u_idx_day_status	insert_time, order_status, expire_time	Unique	BTREE
idx_order_no	order_no	Normal	BTREE
idx_expire_time	expire_time	Normal	BTREE
order_note	order_note(14)	Normal	BTREE

则可以使用如下SQL语句进行查询

```
SELECT
    t. NAME AS d_t_name,
    i. NAME AS i_name,
    i.type AS i_type,
    i.N_FIELDS AS i_column_numbers,
    f. NAME AS i_column_name,
    f.pos AS i_position
FROM
    INNODB_SYS_TABLES AS t
JOIN INNODB_SYS_INDEXES AS i ON t.TABLE_ID = i.TABLE_ID
LEFT JOIN INNODB_SYS_FIELDS AS f ON i.INDEX_ID = f.INDEX_ID
WHERE
    t. NAME = 'lijin/order_exp';
```

信息	结果1	概况	状态		
d_t_name	i_name	i_type	i_column_num	i_column_name	i_position
lijin/order_exp	PRIMARY	3	1	id	0
lijin/order_exp	u_idx_day_status	2	3	insert_time	0
lijin/order_exp	u_idx_day_status	2	3	order_status	1
lijin/order_exp	u_idx_day_status	2	3	expire_time	2
lijin/order_exp	idx_order_no	0	1	order_no	0
lijin/order_exp	idx_expire_time	0	1	expire_time	0
lijin/order_exp	order_note	0	1	order_note	0

结果中的列都很好理解，唯一需要额外解释的是i_type(INNODB_SYS_INDEXES.type)，它是表示索引类型的数字ID：

0 = 二级索引

1 = 集群索引

2 = 唯一索引

3 = 主键索引

32 = 全文索引

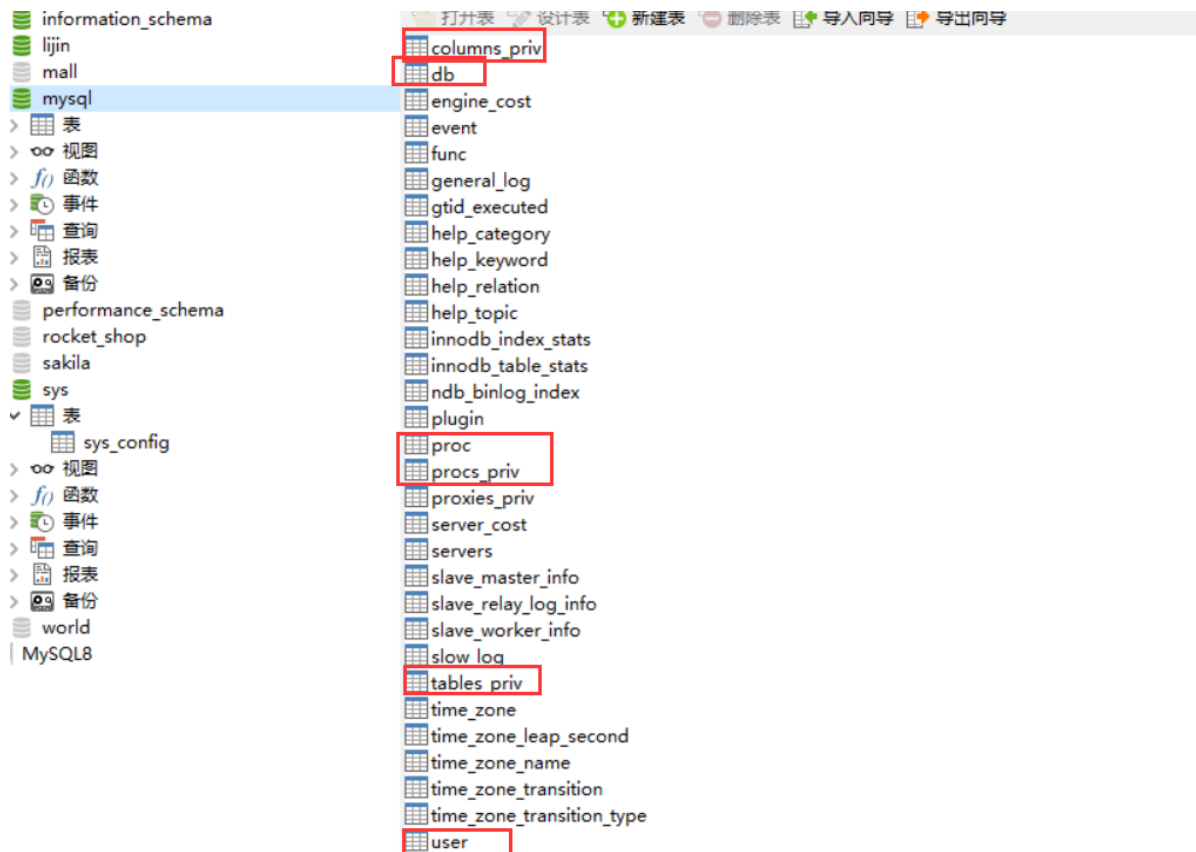
64 = 空间索引

128 = 包含虚拟生成列的二级索引。

1.5.Mysql中mysql系统库

1.5.1.权限系统表

因为权限管理是DBA的职责，所以对于这个部分的表，我们大概了解下即可。在mysql系统库中，MySQL访问权限系统表，放在mysql库中，主要包含如下几个表。



- user: 包含用户账户、全局权限和其他非权限列表（安全配置字段和资源控制字段）。
- db: 数据库级别的权限表。该表中记录的权限信息代表用户是否可以使用这些权限来访问被授予访问的数据库下的所有对象（表或存储程序）。
- tables_priv: 表级别的权限表。
- columns_priv: 字段级别的权限表。
- procs_priv: 存储过程和函数权限表。
- proxies_priv: 代理用户权限表。

提示:

要更改权限表的内容，应该使用账号管理语句（如： CREATE USER 、 GRANT 、 REVOKE等）来间接修改，不建议直接使用DML语句修改权限表。 ****

(grant, revoke语句执行后会变更权限表中相关记录，同时会更新内存中记录用户权限的相关对象。dml语句直接修改权限表只是修改了表中权限信息，需要执行flush privileges;来更新内存中保存用户权限的相关对象)

1.5.2.统计信息表

持久化统计功能是通过将内存中的统计数据存储到磁盘中，使其在数据库重启时可以快速重新读入这些统计信息而不用重新执行统计，从而使得查询优化器可以利用这些持久化的统计信息准确地选择执行计划（如果没有这些持久化的统计信息，那么数据库重启之后内存中的统计信息将会丢失，下一次访问到某库某表时，需要重新计算统计信息，并且重新计算可能会因为估算值的差异导致查询计划发生变更，从而导致查询性能发生变化）。

如何启用统计信息的持久化功能呢？当innodb_stats_persistent = ON时全局的开启统计信息的持久化功能，默认是开启的，

```
show variables like 'innodb_stats_persistent';
```

如果要单独关闭某个表的持久化统计功能，则可以通过ALTER TABLE tbl_name STATS_PERSISTENT = 0语句来修改。

1.5.2.1.innodb_table_stats

innodb_table_stats表提供查询与表数据相关的统计信息。

```
select * from innodb_table_stats where table_name = 'order_exp'\G
```

```
mysql> select * from innodb_table_stats where table_name = 'order_exp'\G
***** 1. row *****
      database_name: lijin
      table_name: order_exp
      last_update: 2022-06-14 13:22:42
      n_rows: 0
      clustered_index_size: 1
      sum_of_other_index_sizes: 4
***** 2. row *****
      database_name: mysql
      table_name: order_exp
      last_update: 2022-05-13 18:30:50
      n_rows: 10621
      clustered_index_size: 97
      sum_of_other_index_sizes: 74
2 rows in set (0.00 sec)
```

database_name: 数据库名称。

- table_name: 表名、分区名或子分区名。
- last_update: 表示InnoDB上次更新统计信息行的时间。
- n_rows: 表中的估算数据记录行数。
- clustered_index_size: 主键索引的大小，以页为单位的估算数值。
- sum_of_other_index_sizes: 其他（非主键）索引的总大小，以页为单位的估算数值。

1.5.2.2.innodb_index_stats

innodb_index_stats表提供查询与索引相关的统计信息。

```
select * from innodb_index_stats where table_name = 'order_exp';
```

```
mysql> select * from innodb_index_stats where table_name = 'order_exp';
```

database_name	table_name	index_name	last_update	stat_name	stat_value	sample_size	stat_description
lijin	order_exp	PRIMARY	2022-06-14 13:22:42	n_diff_pfx01	0	1	id
lijin	order_exp	PRIMARY	2022-06-14 13:22:42	n_leaf_pages	1	NULL	Number of leaf pages in the index
lijin	order_exp	PRIMARY	2022-06-14 13:22:42	size	1	NULL	Number of pages in the index
lijin	order_exp	idx_expire_time	2022-06-14 13:22:42	n_diff_pfx01	0	1	expire_time
lijin	order_exp	idx_expire_time	2022-06-14 13:22:42	n_diff_pfx02	0	1	expire_time,id
lijin	order_exp	idx_expire_time	2022-06-14 13:22:42	n_leaf_pages	1	NULL	Number of leaf pages in the index
lijin	order_exp	idx_expire_time	2022-06-14 13:22:42	size	1	NULL	Number of pages in the index
lijin	order_exp	idx_order_no	2022-06-14 13:22:42	n_diff_pfx01	0	1	order_no
lijin	order_exp	idx_order_no	2022-06-14 13:22:42	n_diff_pfx02	0	1	order_no,id
lijin	order_exp	idx_order_no	2022-06-14 13:22:42	n_leaf_pages	1	NULL	Number of leaf pages in the index
lijin	order_exp	idx_order_no	2022-06-14 13:22:42	size	1	NULL	Number of pages in the index
lijin	order_exp	order_note	2022-06-14 13:22:42	n_diff_pfx01	0	1	order_note
lijin	order_exp	order_note	2022-06-14 13:22:42	n_diff_pfx02	0	1	order_note,id
lijin	order_exp	order_note	2022-06-14 13:22:42	n_leaf_pages	1	NULL	Number of leaf pages in the index
lijin	order_exp	order_note	2022-06-14 13:22:42	size	1	NULL	Number of pages in the index
lijin	order_exp	u_idx_day_status	2022-06-14 13:22:42	n_diff_pfx01	0	1	insert_time
lijin	order_exp	u_idx_day_status	2022-06-14 13:22:42	n_diff_pfx02	0	1	insert_time,order_status
lijin	order_exp	u_idx_day_status	2022-06-14 13:22:42	n_diff_pfx03	0	1	insert_time,order_status,expire_time
lijin	order_exp	u_idx_day_status	2022-06-14 13:22:42	n_leaf_pages	1	NULL	Number of leaf pages in the index
lijin	order_exp	u_idx_day_status	2022-06-14 13:22:42	size	1	NULL	Number of pages in the index
mysql	order_exp	PRIMARY	2022-05-13 18:30:50	n_diff_pfx01	10621	20	id

表字段含义如下。

- database_name: 数据库名称。
- table_name: 表名、分区表名、子分区表名。
- index_name: 索引名称。
- last_update: 表示InnoDB上次更新统计信息行的时间。
- stat_name: 统计信息名称，其对应的统计信息值保存在stat_value字段中。
- stat_value: 保存统计信息名称stat_name字段对应的统计信息值。
- sample_size: stat_value字段中提供的统计信息估计值的采样页数。

- stat_description: 统计信息名称stat_name字段中指定的统计信息的说明。

从表的查询数据中可以看到:

- stat_name字段一共有如下几个统计值。
 - size: 当stat_name字段为size值时, stat_value字段值表示索引中的总页数量。
 - n_leaf_pages: 当stat_name字段为n_leaf_pages值时, stat_value字段值表示索引叶子页的数量。
 - n_diff_pfxNN: NN代表数字(例如01、02等)。当stat_name字段为n_diff_pfxNN值时, stat_value字段值表示索引的first column(即索引的最前索引列, 从索引定义顺序的第一个列开始)列的唯一值数量。例如: 当NN为01时, stat_value字段值就表示索引的第一个列的唯一值数量; 当NN为02时, stat_value字段值就表示索引的第一个和第二个列组合的唯一值数量, 依此类推。此外, 在stat_name = n_diff_pfxNN的情况下, stat_description字段显示一个以逗号分隔的计算索引统计信息字段的列表。
- 从index_name字段值为PRIMARY数据行的stat_description字段的描述信息“id”中可以看出, 主键索引的统计信息只包括创建主键索引时显式指定的列。
- 从index_name字段值为u_idx_day_status数据行的stat_description字段的描述信息“insert_time,order_status,expire_time”中可以看出, 唯一索引的统计信息只包括创建唯一索引时显式指定的列。
- 从index_name字段值为idx_order_no数据行的stat_description字段的描述信息“order_no,id”中可以看出, 普通索引(非唯一的辅助索引)的统计信息包括了显式定义的列和主键列。

注意, 上述的描述中出现的诸如叶子页, 索引的最前索引列等等, 这些东西在索引章节有讲解, 这里不再阐述。

1.5.3.日志记录表

MySQL的日志系统包含: 普通查询日志、慢查询日志、错误日志(记录服务器启动时、运行中、停止时的错误信息)、二进制日志(记录服务器运行过程中数据变更的逻辑日志)、中继日志(记录从库I/O线程从主库获取的主库数据变更日志)、DDL日志(记录DDL语句执行时的元数据变更信息。在MySQL 5.7中只支持写入文件中, 在MySQL 8.0中支持写入innodb_ddl_log表中。在MySQL 5.7中, 只有普通查询日志、慢查询日志支持写入表中(也支持写入文件中), 可以通过log_output=TABLE设置保存到mysql.general_log表和mysql.slow_log表中, 其他日志类型在MySQL 5.7中只支持写入文件中。

1.5.3.1. general_log

general_log表提供查询普通SQL语句的执行记录信息, 用于查看客户端到底在服务器上执行了什么SQL语句。

缺省不开启

```
show variables like 'general_log';
```

```
mysql> show variables like 'general_log';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| general_log   | OFF   |
+-----+-----+
1 row in set (0.01 sec)
```

开启

```
set global log_output='TABLE'; -- 'TABLE,FILE'表示同时输出到表 and 文件
set global general_log=on;
show variables like 'general_log';
```

```
mysql> set global log_output='TABLE';
Query OK, 0 rows affected (0.00 sec)

mysql> set global general_log=on;
Query OK, 0 rows affected (0.00 sec)

mysql> show variables like 'general_log';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| general_log   | ON    |
+-----+-----+
1 row in set (0.00 sec)
```

任意执行一个查询后

```
mysql> select * from order_exp;
+-----+-----+
| id | order_no | order_note |
+-----+-----+
| 16 | DD00_15S | 你好, 李焕英。7排15号,过期时长:DD00_15S |
| 15 | 2021-03-22 18:23:57 | 0 |
| 21 | DD00_10S | 你好, 李焕英。7排10号,过期时长:DD00_10S |
+-----+-----+
```

```
select * from mysql.general_log\G
```

```
mysql> select * from mysql.general_log\G
***** 1. row *****
event_time: 2022-06-14 13:38:13.270999
user_host: root[root] @ localhost []
thread_id: 2
server_id: 1
command_type: Query
argument: show variables like 'general_log'
***** 2. row *****
event_time: 2022-06-14 13:38:36.423902
user_host: root[root] @ localhost []
thread_id: 2
server_id: 1
command_type: Query
argument: select * from mysql.general_log
***** 3. row *****
event_time: 2022-06-14 13:38:49.527102
user_host: root[root] @ localhost []
thread_id: 2
server_id: 1
command_type: Query
argument: select * from order_exp
***** 4. row *****
event_time: 2022-06-14 13:39:01.773540
user_host: root[root] @ localhost []
thread_id: 2
server_id: 1
command_type: Query
argument: select * from mysql.general_log
5 rows in set (0.00 sec)
```

1.5.3.2. slow_log

slow_log表提供查询执行时间超过long_query_time设置值的SQL语句、未使用索引的语句（需要开启参数log_queries_not_using_indexes=ON）或者管理语句（需要开启参数log_slow_admin_statements=ON）。

```
show variables like 'log_queries_not_using_indexes';
show variables like 'log_slow_admin_statements';
```

```
mysql> show variables like 'log_queries_not_using_indexes';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| log_queries_not_using_indexes | OFF |
+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> show variables like 'log_queries_not_using_indexes';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| log_queries_not_using_indexes | OFF |
+-----+-----+
1 row in set (0.00 sec)

mysql> show variables like 'log_slow_admin_statements';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| log_slow_admin_statements | OFF |
+-----+-----+
1 row in set (0.01 sec)
```

开启

```
set global log_queries_not_using_indexes=on;
set global log_slow_admin_statements=on;
show variables like 'log_queries_not_using_indexes';
show variables like 'log_slow_admin_statements';
```

```
mysql> set global log_queries_not_using_indexes=on;
Query OK, 0 rows affected (0.00 sec)

mysql> set global log_slow_admin_statements=on;
Query OK, 0 rows affected (0.00 sec)

mysql> show variables like 'log_queries_not_using_indexes';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| log_queries_not_using_indexes | ON |
+-----+-----+
1 row in set (0.00 sec)

mysql> show variables like 'log_slow_admin_statements';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| log_slow_admin_statements | ON |
+-----+-----+
1 row in set (0.00 sec)
```

我们已经知道慢查询日志可以帮助定位可能存在问题的SQL语句，从而进行SQL语句层面的优化。但是默认值为关闭的，需要我们手动开启。

```
show VARIABLES like 'slow_query_log';
```

```
1 show VARIABLES like 'slow_query_log';
2
```

信息

结果1

概况

状态

Variable_name	Value
slow_query_log	ON

```
set GLOBAL slow_query_log=1;
```

开启1，关闭0

但是多慢算慢？MySQL中可以设定一个阈值，将运行时间超过该值的所有SQL语句都记录到慢查询日志中。long_query_time参数就是这个阈值。默认值为10，代表10秒。

```
show VARIABLES like '%long_query_time%';
```

当然也可以设置

```
set global long_query_time=0;
```

默认10秒，这里为了演示方便设置为0

```
mysql> show VARIABLES like 'slow_query_log';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| slow_query_log | OFF   |
+-----+-----+
1 row in set (0.00 sec)

mysql> set GLOBAL slow_query_log=1;
Query OK, 0 rows affected (0.00 sec)

mysql> show VARIABLES like 'slow_query_log';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| slow_query_log | ON    |
+-----+-----+
1 row in set (0.00 sec)

mysql> set global long_query_time=0;
Query OK, 0 rows affected (0.00 sec)
```

然后我们测试一把，随便写一个SQL

```
mysql> SELECT * FROM order_exp where expire_duration =13;
+-----+-----+-----+-----+-----+
| id | order_no | order_note | insert_time | expire_time |
+-----+-----+-----+-----+-----+
| 130 | DD00_13S | 你好, 李焕英。7排13号,过期时长:DD00_13S | 2021-03-22 18:34:56 | 2021-03-22 18:35:09 |
| 151 | DD00_13S | 你好, 李焕英。7排13号,过期时长:DD00_13S | 2021-03-22 18:34:57 | 2021-03-22 18:35:10 |
+-----+-----+-----+-----+-----+
```

```
select * from mysql.slow_log\G
```

```
mysql> select * from mysql.slow_log\G
***** 1. row *****
start_time: 2022-06-14 13:50:23.836579
user_host: root[root] @ localhost []
query_time: 00:00:00.009696
lock_time: 00:00:00.000254
rows_sent: 17
rows_examined: 10567
db: mysql
last_insert_id: 0
insert_id: 0
server_id: 1
sql_text: SELECT * FROM order_exp where expire_duration =13
thread_id: 2
1 row in set (0.00 sec)
```

1.5.4.InnoDB中的统计数据

我们前边唠叨查询成本的时候经常用到一些统计数据，比如通过SHOW TABLE STATUS可以看到关于表的统计数据，通过SHOW INDEX可以看到关于索引的统计数据，那么这些统计数据是怎么来的呢？它们是以什么方式收集的呢？

1.5.4.1 统计数据存储方式

InnoDB提供了两种存储统计数据的方式：

永久性的统计数据，这种统计数据存储在磁盘上，也就是服务器重启之后这些统计数据还在。

非永久性的统计数据，这种统计数据存储在内存中，当服务器关闭时这些统计数据就都被清除了，等到服务器重启之后，在某些适当的场景下才会重新收集这些统计数据。

MySQL给我们提供了系统变量innodb_stats_persistent来控制到底采用哪种方式去存储统计数据。在MySQL 5.6.6之前，innodb_stats_persistent的值默认是OFF，也就是说InnoDB的统计数据默认是存储到内存的，之后的版本中innodb_stats_persistent的值默认是ON，也就是统计数据默认被存储到磁盘上。

```
SHOW VARIABLES LIKE 'innodb_stats_persistent';
```

```
mysql> SHOW TABLES FROM mysql LIKE 'innodb%';
+-----+
| Tables_in_mysql (innodb%) |
+-----+
| innodb_index_stats         |
| innodb_table_stats         |
+-----+
2 rows in set (0.00 sec)
```

不过最近的MySQL版本都基本不用基于内存的非永久性统计数据了，所以我们也就不深入研究。

不过InnoDB默认是以表为单位来收集和存储统计数据的，也就是说我们可以把某些表的统计数据（以及该表的索引统计数据）存储在磁盘上，把另一些表的统计数据存储在内存中。怎么做到的呢？我们可以在创建和修改表的时候通过指定STATS_PERSISTENT属性来指明该表的统计数据存储方式：

CREATE TABLE 表名 (...)

Engine=InnoDB, STATS_PERSISTENT = (1|0);

ALTER TABLE 表名

Engine=InnoDB, STATS_PERSISTENT = (1|0);

当STATS_PERSISTENT=1时，表明我们想把该表的统计数据永久的存储到磁盘上，当

STATS_PERSISTENT=0时，表明我们想把该表的统计数据临时的存储到内存中。如果我们在创建表时未指定STATS_PERSISTENT属性，那默认采用系统变量innodb_stats_persistent的值作为该属性的值。

1.5.4.2 基于磁盘的永久性统计数据

当我们选择把某个表以及该表索引的统计数据存放到磁盘上时，实际上是把这些统计数据存储到了两个表里：

```
SHOW TABLES FROM mysql LIKE 'innodb%';
```

```
mysql> SHOW TABLES FROM mysql LIKE 'innodb%';
+-----+
| Tables_in_mysql (innodb%) |
+-----+
| innodb_index_stats         |
| innodb_table_stats         |
+-----+
2 rows in set (0.00 sec)
```

可以看到，这两个表都位于mysql系统数据库下边，其中：

innodb_table_stats存储了关于表的统计数据，每一条记录对应着一个表的统计数据。

innodb_index_stats存储了关于索引的统计数据，每一条记录对应着一个索引的一个统计项的统计数据。

innodb_table_stats

直接看一下这个innodb_table_stats表中的各个列都是干嘛的：

```
mysql> desc mysql.innodb_table_stats;
+-----+-----+
| Field | Type |
+-----+-----+
| database_name | varchar(64) |
| table_name | varchar(199) |
| last_update | timestamp |
| n_rows | bigint(20) |
| clustered_index_size | bigint(20) |
| sum_of_other_index_sizes | bigint(20) |
+-----+-----+
6 rows in set (0.00 sec)
```

database_name 数据库名

table_name 表名

last_update 本条记录最后更新时间

n_rows表中记录的条数

clustered_index_size 表的聚簇索引占用的页面数量

sum_of_other_index_sizes 表的其他索引占用的页面数量

我们直接看一下这个表里的内容：

```
SELECT * FROM mysql.innodb_table_stats;
```

```
mysql> SELECT * FROM mysql.innodb_table_stats;
+-----+-----+-----+-----+-----+-----+
| database_name | table_name | last_update | n_rows | clustered_index_size | sum_of_other_index_sizes |
+-----+-----+-----+-----+-----+-----+
| mysql | gtid_executed | 2021-03-19 16:19:15 | 0 | 1 | 0 |
| mysqladv | e1 | 2021-03-31 16:34:58 | 3 | 1 | 0 |
| mysqladv | e2 | 2021-03-31 16:35:08 | 3 | 1 | 0 |
| mysqladv | order_exp | 2021-03-31 15:31:28 | 10248 | 97 | 81 |
| mysqladv | s1 | 2021-03-26 21:42:50 | 10573 | 161 | 50 |
| mysqladv | s2 | 2021-03-26 18:11:20 | 10315 | 97 | 69 |
| mysqladv | score | 2021-03-31 16:50:57 | 4 | 1 | 0 |
| mysqladv | student | 2021-03-31 16:48:12 | 4 | 1 | 0 |
| mysqladv | teacher | 2021-03-30 20:11:12 | 5 | 1 | 1 |
| mysqladv | test | 2021-03-22 14:31:58 | 1 | 1 | 0 |
| mysqladv | undo_test | 2021-03-29 21:53:16 | 0 | 1 | 1 |
| sys | sys_config | 2021-03-19 16:19:16 | 6 | 1 | 0 |
+-----+-----+-----+-----+-----+-----+
12 rows in set (0.00 sec)
```

几个重要统计信息项的值如下：

n_rows的值是10350，表明order_exp表中大约有10350条记录，注意这个数据是估计值。

clustered_index_size的值是97，表明order_exp表的聚簇索引占用97个页面，这个值也是也是一个估计值。

sum_of_other_index_sizes的值是81，表明order_exp表的其他索引一共占用81个页面，这个值也是也是一个估计值。

n_rows统计项的收集

InnoDB统计一个表中有多少行记录是这样的：

按照一定算法（并不是纯粹随机的）选取几个叶子节点页面，计算每个页面中主键值记录数量，然后计算平均一个页面中主键值的记录数量乘以全部叶子节点的数量就算是该表的n_rows值。

可以看出来这个n_rows值精确与否取决于统计时采样的页面数量，MySQL用名为innodb_stats_persistent_sample_pages的系统变量来控制使用永久性的统计数据时，计算统计数据时采样的页面数量。该值设置的越大，统计出的n_rows值越精确，但是统计耗时也就最久；该值设置的越小，统计出的n_rows值越不精确，但是统计耗时特别少。所以在实际使用是需要我们去权衡利弊，该系统变量的默认值是20。

InnoDB默认是以表为单位来收集和存储统计数据的，我们也可以单独设置某个表的采样页面的数量，设置方式就是在创建或修改表的时候通过指定STATS_SAMPLE_PAGES属性来指明该表的统计数据存储方式：

```
CREATE TABLE 表名 (...)
```

```
Engine=InnoDB, STATS_SAMPLE_PAGES = 具体的采样页面数量;
```

```
ALTER TABLE 表名
```

```
Engine=InnoDB, STATS_SAMPLE_PAGES = 具体的采样页面数量;
```

如果我们在创建表的语句中并没有指定STATS_SAMPLE_PAGES属性的话，将默认使用系统变量innodb_stats_persistent_sample_pages的值作为该属性的值。

clustered_index_size和sum_of_other_index_sizes统计项的收集牵涉到很具体的InnoDB表空间的知识 and 存储页面数据的细节，我们就不深入讲解了。

innodb_index_stats

直接看一下这个innodb_index_stats表中的各个列都是干嘛的：

```
desc mysql.innodb_index_stats;
```

字段名描述

database_name 数据库名

table_name 表名

index_name 索引名

last_update 本条记录最后更新时间

stat_name 统计项的名称

stat_value 对应的统计项的值

sample_size 为生成统计数据而采样的页面数量

stat_description 对应的统计项的描述

innodb_index_stats表的每条记录代表着一个索引的一个统计项。可能这会大家有些懵逼这个统计项到底指什么，别着急，我们直接看一下关于order_exp表的索引统计数据都有些什么：

```
SELECT * FROM mysql.innodb_index_stats WHERE table_name = 'order_exp';
```

```
mysql> SELECT * FROM mysql.innodb_index_stats WHERE table_name = 'order_exp';
```

database_name	table_name	index_name	last_update	stat_name	stat_value	sample_size	stat_description
mysqladv	order_exp	PRIMARY	2021-03-25 23:32:02	n_diff_pfx01	9460	20	id
mysqladv	order_exp	PRIMARY	2021-03-25 23:32:02	n_leaf_pages	79	NULL	Number of leaf pages in the index
mysqladv	order_exp	PRIMARY	2021-03-25 23:32:02	size	97	NULL	Number of pages in the index
mysqladv	order_exp	idx_expire_time	2021-03-31 15:31:28	n_diff_pfx01	9802	13	expire_time
mysqladv	order_exp	idx_expire_time	2021-03-31 15:31:28	n_diff_pfx02	10562	13	expire_time,id
mysqladv	order_exp	idx_expire_time	2021-03-31 15:31:28	n_leaf_pages	13	NULL	Number of leaf pages in the index
mysqladv	order_exp	idx_expire_time	2021-03-31 15:31:28	size	14	NULL	Number of pages in the index
mysqladv	order_exp	idx_order_no	2021-03-25 23:32:02	n_diff_pfx01	9432	24	order_no
mysqladv	order_exp	idx_order_no	2021-03-25 23:32:02	n_diff_pfx02	9774	24	order_no,id
mysqladv	order_exp	idx_order_no	2021-03-25 23:32:02	n_leaf_pages	24	NULL	Number of leaf pages in the index
mysqladv	order_exp	idx_order_no	2021-03-25 23:32:02	size	25	NULL	Number of pages in the index
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	n_diff_pfx01	915	27	insert_time
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	n_diff_pfx02	915	27	insert_time,order_status
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	n_diff_pfx03	9774	27	insert_time,order_status,expire_time
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	n_leaf_pages	27	NULL	Number of leaf pages in the index
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	size	28	NULL	Number of pages in the index

16 rows in set (0.00 sec)

先查看index_name列，这个列说明该记录是哪个索引的统计信息，从结果中我们可以看出来，PRIMARY索引（也就是主键）占了3条记录，idx_expire_time索引占了6条记录。

针对index_name列相同的记录，stat_name表示针对该索引的统计项名称，stat_value展示的是该索引在该统计项上的值，stat_description指的是来描述该统计项的含义的。我们来具体看一下一个索引都有哪些统计项：

n_leaf_pages：表示该索引的叶子节点占用多少页面。

size：表示该索引共占用多少页面。

n_diff_pfxNN：表示对应的索引列不重复的值有多少。其中的NN长得有点儿怪呀，啥意思呢？

其实NN可以被替换为01、02、03... 这样的数字。比如对于u_idx_day_status来说：

n_diff_pfx01表示的是统计insert_time这单一个列不重复的值有多少。

n_diff_pfx02表示的是统计insert_time,order_status这两个列组合起来不重复的值有多少。

n_diff_pfx03表示的是统计insert_time,order_status,expire_time这三个列组合起来不重复的值有多少。

n_diff_pfx04表示的是统计key_pare1、key_pare2、expire_time、id这四个列组合起来不重复的值有多少。

对于普通的二级索引，并不能保证它的索引列值是唯一的，比如对于idx_order_no来说，key1列就可能有很多值重复的记录。此时只有在索引列上加上主键值才可以区分两条索引列值都一样的二级索引记录。

对于主键和唯一二级索引则没有这个问题，它们本身就可以保证索引列值的不重复，所以也不需要再统计一遍在索引列后加上主键值的不重复值有多少。比如u_idx_day_statu和idx_order_no。

在计算某些索引列中包含多少不重复值时，需要对一些叶子节点页面进行采样，sample_size列就表明了采样的页面数量是多少。

对于有多个列的联合索引来说，采样的页面数量是：innodb_stats_persistent_sample_pages × 索引列的个数。

```
mysql> show variables like 'innodb_stats_transient_sample_pages';
```

Variable_name	Value
innodb_stats_transient_sample_pages	8

1 row in set (0.00 sec)

当需要采样的页面数量大于该索引的叶子节点数量的话，就直接采用全表扫描来统计索引列的不重复值数量了。所以大家可以在查询结果中看到不同索引对应的size列的值可能是不同的。

定期更新统计数据

随着我们不断的对表进行增删改操作，表中的数据也一直在变化，innodb_table_stats和innodb_index_stats表里的统计数据也在变化。MySQL提供了如下两种更新统计数据的方式：

开启innodb_stats_auto_recalc。

系统变量innodb_stats_auto_recalc决定着服务器是否自动重新计算统计数据，它的默认值是ON，也就是该功能默认是开启的。每个表都维护了一个变量，该变量记录着对该表进行增删改的记录条数，如果发生变动的记录数量超过了表大小的10%，并且自动重新计算统计数据的功能是打开的，那么服务器会重新进行一次统计数据的计算，并且更新innodb_table_stats和innodb_index_stats表。不过自动重新计算统计数据的过程是异步发生的，也就是即使表中变动的记录数超过了10%，自动重新计算统计数据也不会立即发生，可能会延迟几秒才会进行计算。

再一次强调，InnoDB默认是以表为单位来收集和存储统计数据的，我们也可以单独为某个表设置是否自动重新计算统计数的属性，设置方式就是在创建或修改表的时候通过指定STATS_AUTO_RECALC属性来指明该表的统计数据存储方式：

```
CREATE TABLE 表名 (...)
```

```
Engine=InnoDB, STATS_AUTO_RECALC = (1|0);
```

```
ALTER TABLE 表名
```

```
Engine=InnoDB, STATS_AUTO_RECALC = (1|0);
```

当STATS_AUTO_RECALC=1时，表明我们想让该表自动重新计算统计数据，当STATS_AUTO_RECALC=0时，表明不想让该表自动重新计算统计数据。如果我们在创建表时未指定STATS_AUTO_RECALC属性，那默认采用系统变量innodb_stats_auto_recalc的值作为该属性的值。

手动调用ANALYZE TABLE语句来更新统计信息

如果innodb_stats_auto_recalc系统变量的值为OFF的话，我们也可以手动调用ANALYZE TABLE语句来重新计算统计数据，比如我们可以这样更新关于order_exp表的统计数据：

```
ANALYZE TABLE order_exp;
```

```
mysql> SELECT * FROM mysql.innodb_index_stats WHERE table_name = 'order_exp';
```

database_name	table_name	index_name	last_update	stat_name	stat_value	sample_size	stat_description
mysqladv	order_exp	PRIMARY	2021-03-25 23:32:02	n_diff_pfx01	9460	20	id
mysqladv	order_exp	PRIMARY	2021-03-25 23:32:02	n_leaf_pages	79	NULL	Number of leaf pages in the index
mysqladv	order_exp	PRIMARY	2021-03-25 23:32:02	size	97	NULL	Number of pages in the index
mysqladv	order_exp	idx_expire_time	2021-03-31 15:31:28	n_diff_pfx01	9802	13	expire_time
mysqladv	order_exp	idx_expire_time	2021-03-31 15:31:28	n_diff_pfx02	10562	13	expire_time,id
mysqladv	order_exp	idx_expire_time	2021-03-31 15:31:28	n_leaf_pages	13	NULL	Number of leaf pages in the index
mysqladv	order_exp	idx_expire_time	2021-03-31 15:31:28	size	14	NULL	Number of pages in the index
mysqladv	order_exp	idx_order_no	2021-03-25 23:32:02	n_diff_pfx01	9432	24	order_no
mysqladv	order_exp	idx_order_no	2021-03-25 23:32:02	n_diff_pfx02	9774	24	order_no,id
mysqladv	order_exp	idx_order_no	2021-03-25 23:32:02	n_leaf_pages	24	NULL	Number of leaf pages in the index
mysqladv	order_exp	idx_order_no	2021-03-25 23:32:02	size	25	NULL	Number of pages in the index
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	n_diff_pfx01	915	27	insert_time
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	n_diff_pfx02	915	27	insert_time,order_status
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	n_diff_pfx03	9774	27	insert_time,order_status,expire_time
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	n_leaf_pages	27	NULL	Number of leaf pages in the index
mysqladv	order_exp	u_idx_day_status	2021-03-25 23:32:02	size	28	NULL	Number of pages in the index

```
16 rows in set (0.00 sec)
```

```
mysql> ANALYZE TABLE order_exp;
```

Table	Op	Msg type	Msg text
mysqladv.order_exp	analyze	status	OK

```
1 row in set (0.02 sec)
```

```
mysql> SELECT * FROM mysql.innodb_index_stats WHERE table_name = 'order_exp';
```

database_name	table_name	index_name	last_update	stat_name	stat_value	sample_size	stat_description
mysqladv	order_exp	PRIMARY	2021-04-01 00:05:51	n_diff_pfx01	10345	20	id
mysqladv	order_exp	PRIMARY	2021-04-01 00:05:51	n_leaf_pages	86	NULL	Number of leaf pages in the index
mysqladv	order_exp	PRIMARY	2021-04-01 00:05:51	size	97	NULL	Number of pages in the index
mysqladv	order_exp	idx_expire_time	2021-04-01 00:05:51	n_diff_pfx01	9802	13	expire_time
mysqladv	order_exp	idx_expire_time	2021-04-01 00:05:51	n_diff_pfx02	10562	13	expire_time,id

ANALYZE TABLE语句会立即重新计算统计数据，也就是这个过程是同步的，在表中索引多或者采样页面特别多时这个过程可能会特别慢最好在业务不是很繁忙的时候再运行。

手动更新innodb_table_stats和innodb_index_stats表

其实innodb_table_stats和innodb_index_stats表就相当于一个普通的表一样，我们能对它们做增删改查操作。这也就意味着我们可以手动更新某个表或者索引的统计数据。比如说我们想把order_exp表关于行数的统计数据更改一下可以这么做：

步骤一：更新innodb_table_stats表。

步骤二：让MySQL查询优化器重新加载我们更改过的数据。

更新完innodb_table_stats只是单纯的修改了一个表的数据，需要让MySQL查询优化器重新加载我们更改过的数据，运行下边的命令就可以了：

```
FLUSH TABLE order_exp;
```